# Relatório Técnico — Motor de Busca com PLN

**Projeto:** busca textual em um catálogo de imagens de sensoriamento remoto
**Foco:** pipeline clássico de PLN, Bag of Words, TF-IDF e similaridade de cosseno
**Tecnologias:** Python, Flask, PostgreSQL, NLTK, scikit-learn, rapidfuzz
**Restrição:** sem LLM, sem embeddings neurais e sem API externa

> **Ideia central:** o sistema não "interpreta" a pergunta com um modelo de linguagem. Ele transforma a pergunta em um vetor de palavras, compara esse vetor com perguntas de referência e escolhe a intenção mais parecida. Depois executa a consulta SQL correspondente.

---

## 1. Resumo

Este trabalho implementa um motor de busca em página única para um catálogo de imagens de sensoriamento remoto. O usuário digita uma pergunta em português ("quantas imagens existem?", "imagens de drone em 2024") e o sistema responde com texto e, quando faz sentido, com os cartões das imagens.

A parte de PLN é o coração do projeto. O pipeline executa, nesta ordem:

1. normalização (minúsculas, sem acento, sem pontuação);
2. correção ortográfica contra um léxico fechado do domínio;
3. canonização de sinônimos de entidades e de intenção;
4. substituição de anos e datas por tokens semânticos;
5. tokenização;
6. remoção de stopwords, preservando palavras que carregam intenção;
7. stemming com RSLP;
8. representação Bag of Words com pesos TF-IDF;
9. similaridade de cosseno contra 134 paráfrases de referência.

As paráfrases estão organizadas em **20 intenções**. Cada intenção conhece a consulta SQL que deve ser executada e quais valores (categoria, origem, ano ou data) precisam ser extraídos da pergunta.

**Resultado principal:** no conjunto de 113 perguntas de avaliação, o sistema acertou a intenção em 100% dos casos. O conjunto foi escrito pela própria equipe durante o desenvolvimento, portanto **não é um benchmark cego**. O valor dele é permitir a ablação: mostramos quanto cada etapa de PLN contribui. A etapa mais crítica é a canonização de sinônimos; sem ela, a acurácia cai para 55,9%.

---

## 2. O problema e a abordagem

O enunciado pede uma aplicação que receba uma consulta textual simples e a mapeie para uma consulta SQL sobre metadados de imagens. Há, portanto, **duas tarefas diferentes**:

| Tarefa | Pergunta típica | O que precisa descobrir |
|---|---|---|
| Reconhecer a intenção | "Quantas imagens de vegetação existem?" | contar, filtrar por categoria |
| Extrair os valores | "Imagens de drone em 2024" | origem = drone, ano = 2024 |

O projeto separa essas tarefas de propósito:

> **O Bag of Words decide a INTENÇÃO. A extração de entidades decide os VALORES da SQL.**

Essa separação resolve um problema clássico do BOW: ele perde a ordem das palavras e não distingue números. Se `2024` fosse apenas mais um token, "imagens de 2023" e "imagens de 2024" teriam representações praticamente idênticas e a SQL sairia com o ano errado. Por isso, antes da vetorização, o sistema troca o ano por `ano` e guarda o valor concreto ( `2024` ) em uma estrutura separada.

---

## 3. Arquitetura

```
Navegador (HTML/CSS/JS)
        │  POST /api/busca {"pergunta": "..."}
        ▼
Flask (app.py)
        │
        ├──► Pipeline de PLN (pln.py)
        │      normalização → correção → sinônimos → números
        │      → tokens → stopwords → stemming → BOW
        │
        ├──► Catálogo de intenções (intencoes.py)
        │      20 intenções, 134 paráfrases, SQL parametrizada
        │
        ├──► Motor (motor.py)
        │      TF-IDF → cosseno → ranking → regras de entidades
        │
        └──► PostgreSQL (metadata_table)
               executa a SQL e devolve as linhas
        │
        ▼
JSON → mensagem, cartões de imagem e painel de debug
```

A separação permite testar o pipeline de PLN sem Flask e sem banco. Os scripts `demo_pln.py` , `demo_bow.py` , `testar_parafrases.py` e `avaliacao.py` fazem exatamente isso.

A infraestrutura é intencionalmente simples: PostgreSQL armazena os metadados, Flask expõe a API e o front-end é uma página única. O foco do trabalho está no motor de PLN, não na camada web.

---

# 4. O pipeline de PLN

## 4.1 Visão geral e ordem das etapas

| Ordem | Etapa | Função | Arquivo |
|---|---|---|---|
| 1 | Normalização | minúsculas, sem acento, sem pontuação | `texto.py` |
| 2 | Correção ortográfica | distância de edição contra léxico do domínio | `corretor.py` |
| 3 | Canonização de entidades | `mata`, `floresta` → `vegetacao` | `entidades.py` |
| 4 | Canonização de intenção | `quantas` → `contar`; `mostre` → `listar` | `lexico.py` / `pln.py` |
| 5 | Números e datas | `2024` → `ano`; `12/03/2024` → `data` | `pln.py` |
| 6 | Tokenização | separar por espaços, descartar tokens de 1 letra | `pln.py` |
| 7 | Stopwords | remover palavras vazias, preservar intenção | `pln.py` |
| 8 | Stemming RSLP | `imagens` → `imag`; `vegetação` → `vegetaca` | `pln.py` |
| 9 | BOW / TF-IDF | representação vetorial | `motor.py` |
| 10 | Cosseno | comparação com as referências | `motor.py` |

A ordem não é arbitrária:

- a **correção** vem antes da canonização porque `drono` precisa virar `drone` antes de ser reconhecido como origem;
- a **canonização** vem antes do stemming para que sinônimos convirjam para a mesma forma canônica;
- os **números** são substituídos antes da tokenização para não virarem tokens genéricos e para que a entidade concreta seja guardada à parte;
- o **stemming** vem por último porque atua sobre tokens já limpos.

## 4.2 Normalização

Implementada em `texto.py`:

```
def normalizar(texto: str) -> str:
    texto = texto.lower()
    texto = unicodedata.normalize("NFKD", texto)
    texto = "".join(c for c in texto if not unicodedata.combining(c))
    texto = re.sub(r"[^a-z0-9\s/]", " ", texto)
    return re.sub(r"\s+", " ", texto).strip()
```

Exemplo:

```
"Quais imagens possuem vegetação?"
→ "quais imagens possuem vegetacao"
```

A barra `/` é preservada de propósito. Sem ela, `12/03/2024` viraria `12 03 2024` e a expressão regular que extrai a data completa deixaria de funcionar.

## 4.3 Correção ortográfica

O corretor usa `rapidfuzz` (distância de edição de Levenshtein) contra um **léxico fechado do domínio**. O léxico é montado automaticamente a partir de:

- palavras extras de perguntas ( `foto` , `imagem` , `mostre` , `drone` , etc.);
- todas as palavras das paráfrases do catálogo;
- sinônimos de categoria e de origem.

Para evitar correções absurdas, o algoritmo impõe três restrições:

1. a palavra corrigida deve começar com a **mesma letra**;
2. o tamanho deve diferir no máximo 3 caracteres;
3. a similaridade precisa passar de um limite: **73** para palavras com 6 ou mais letras e **85** para palavras curtas.

```
if len(token) < 4:
    return token
candidatos = [
    p for p in LEXICO
    if p[0] == token[0] and abs(len(p) - len(token)) <= 3
]
limite = 73 if len(token) >= 6 else 85
```

Exemplos reais:

| Entrada | Correção |
|---|---|
| `vejetassao` | `vegetacao` |
| `drono` | `drone` |
| `imagems` | `imagens` |
| `sateliti` | `satelite` |

O corretor pode gerar efeitos colaterais gramaticais, como corrigir `esta` para `estao` ou `cadastrado` para `cadastradas`. Isso não prejudica o BOW porque essas palavras são stopwords ou caem no mesmo stem (`cadastr`). É um exemplo de que o pipeline é tolerante a ruído, mas não é um corretor gramatical.

**Limitação:** se o erro estiver na primeira letra (`vejetacao` em vez de `vegetacao`), a restrição de mesma letra inicial impede a correção. Essa limitação está documentada e poderia ser removida com um custo maior de falsos positivos.

## 4.4 Canonização de sinônimos

Existem dois dicionários com propósitos diferentes.

**Entidades** (`entidades.py`): traduzem o vocabulário do usuário para os valores que existem no banco.

| Grupo | Formas aceitas | Forma canônica |
|---|---|---|
| Categoria | vegetacao, mata, floresta, verde, vegetado, bosque | `vegetacao` |
| Categoria | solo exposto, solo, terra, deserto, solo nu, solo descoberto | `solo_exposto` |
| Categoria | agua, rio, lago, lagoa, mar, corpo d agua, represa | `agua` |
| Origem | satelite, orbital, sentinel, landsat, espacial | `satelite` |
| Origem | drone, drono, vant, uav, aerolevantamento | `drone` |

**Intenção** (`lexico.py`): aproximam formas de perguntar que deveriam cair na mesma intenção.

| Grupo | Formas | Forma canônica |
|---|---|---|
| Contar | quantas, quantos, total, numero, ha | `contar` |
| Listar | mostre, liste, exiba, quais, qual, quero ver | `listar` |
| Imagem | imagem, imagens, foto, fotos, fotografia | `imagem` |
| Todos | todas, todos, catalogo, acervo, completo | `todos` |
| Recente | recente, recentes, ultima, ultimas, novo, nova | `recente` |
| Antiga | antiga, antigas, antigo, antigos | `antiga` |
| Capturar | capturada, tirada, feita, obtida, registrada | `capturar` |
| Origem | origem, fontes, sensor, de onde vem | `origem` |

A implementação usa **um único passe de regex**, com os padrões mais longos primeiro:

```
_PADROES = sorted(_MAPA_SINONIMOS, key=len, reverse=True)
_REGEX = re.compile(r"\b(" + "|".join(map(re.escape, _PADROES)) + r")\b")
```

Isso evita substituições em cascata, como `solo` dentro de `solo exposto`.

**Por que essa etapa é a mais importante?** Sem ela, `mata`, `floresta` e `vegetacao` ocupam dimensões diferentes do BOW, e `quantas` não se conecta a `total`. Na ablação, remover a canonização derruba a acurácia de 100% para **55,9%**. É a maior contribuição isolada do pipeline.

## 4.5 Números e datas fora do BOW

Bag of Words é um saco de palavras: ele perde a ordem e não distingue valores. Por isso, anos e datas são extraídos por expressão regular **antes** da vetorização e substituídos por tokens semânticos:

```
"imagens de 2024"        → "imagem de ano"
"imagens de 12/03/2024"  → "imagem de data"
```

O valor concreto é guardado em um dicionário separado:

```
{"categoria": "vegetacao", "origem": "drone", "ano": 2024}
```

Assim, o BOW aprende a **forma** da pergunta ("listar por ano") e a entidade fornece o **valor** que entra na SQL (`2024`). Sem essa etapa, a acurácia cai para 73,5% na ablação.

## 4.6 Tokenização

Após a normalização, a tokenização é uma separação por espaços. Tokens de uma letra são descartados porque não carregam conteúdo:

```
"quais imagens possuem vegetacao"
→ ["quais", "imagens", "possuem", "vegetacao"]
```

## 4.7 Stopwords

A lista base tem 128 palavras. O projeto remove as stopwords vazias, mas **preserva de propósito** palavras que parecem stopwords e carregam intenção neste domínio:

```
quantas, quantos, total, numero, ha, existe, existem,
mostre, mostrar, liste, exiba, quero, ver,
quais, qual, todas, todos, acervo, catalogo,
recente, ultima, novo, nova, categoria, origem, fonte, sensor
```

A implementação é:

```
STOP = STOP_BASE - MANTER
```

O conjunto final tem 124 palavras. A diferença parece pequena, mas é decisiva: sem preservar `quantas` e `quais`, as perguntas "quantas imagens existem" e "quais imagens existem" perdem justamente a palavra que distingue **contar** de

**listar**. Na ablação, remover as stopwords corretamente tratadas derruba a acurácia para 71,6%.

## 4.8 Stemming com RSLP

O stemmer usado é o **RSLP** (Removedor de Sufixos da Língua Portuguesa), do NLTK. Ele reduz a palavra à raiz:

| Palavra | Stem |
|---|---|
| imagens | `imag` |
| imagem | `imag` |
| vegetação | `vegetaca` |
| água | `agu` |
| drone | `dron` |
| satélite | `satelit` |
| existem | `exist` |
| recentes | `recent` |

O stemming unifica formas flexionadas e absorve parte dos erros de digitação. Na ablação, removê-lo derruba a acurácia para 82,4%.

### 4.8.1 Por que stemming e não lematização?

Stemming e lematização reduzem a variação morfológica, mas de formas diferentes:

| Técnica | O que faz | Resultado | Precisa de quê |
|---|---|---|---|
| **Stemming** | corta sufixos por regras | raiz que pode não ser uma palavra | regras linguísticas |
| **Lematização** | devolve a forma dicionarizada | lema válido | léxico + classe gramatical (POS tagging) |

Exemplos reais:

| Palavra | Stem RSLP | Lema esperado | Observação |
|---|---|---|---|
| `imagens` | `imag` | `imagem` | o stem não é palavra, mas agrupa |
| `capturadas` | `captur` | `capturar` | idem |
| `existem` | `exist` | `existir` | idem |
| `satélites` | `satelit` | `satélite` | idem |
| `água` | `agu` | `água` | idem |
| `foi` | `foi` | `ser` ou `ir` | a lematização depende do contexto |

| Palavra | Stem RSLP | Lema esperado | Observação |
|---|---|---|---|
| `mata` | `mat` (fora do pipeline) | `mata` ou `matar` | ambíguo sem contexto |

No pipeline, `mata` nem chega a virar `mat`: a canonização de entidades transforma `mata` em `vegetacao` antes do stemming. O stem final é `vegetaca`.

A escolha pelo RSLP se justifica por cinco motivos:

1. **O BOW não precisa de palavras válidas.** Ele só precisa que variantes da mesma palavra caiam na mesma dimensão do vetor. `imag`, `imagem` e `imagens` não precisam ser bonitos; precisam ser iguais.
2. **É mais leve.** Lematizar em português normalmente exige tokenizador, etiquetador morfológico e léxico. O RSLP é baseado em regras, já vem no NLTK e funciona offline, sem modelo adicional.
3. **Ajuda na robustez a erros de digitação.** Por ser baseado em regras, o RSLP reduz até palavras que não existem no dicionário: `imagems` → `imag`, `drono` → `dron`. Um lematizador baseado em léxico pode não reconhecer `imagems` e devolver a palavra intacta.
4. **É determinístico e explicável.** A mesma palavra sempre gera o mesmo stem, o que combina com a proposta de um sistema reprodutível e auditável.
5. **Está alinhado ao escopo da disciplina.** Stemming é uma das componentes clássicas de pipeline de PLN, e o RSLP é o stemmer específico para português.

O que se perde com essa escolha:

| Limitação | Exemplo | Impacto no projeto |
|---|---|---|
| Gera não-palavras | `vegetação` → `vegetaca` | nenhum para o BOW; apenas estranheza visual |
| Pode juntar palavras diferentes | `mata` e `matar` → `mat` | no pipeline, `mata` é canonizado antes |
| Não desambigua sentido | `banco` (assento) e `banco` (dados) | o domínio restringe o sentido |
| Não devolve forma dicionarizada | `foi` continua `foi` | irrelevante para a intenção |

A lematização seria linguisticamente mais precisa, mas o critério aqui é a qualidade da representação para o BOW, não a correção gramatical. Para gerar texto ou fazer análise sintática, lematização seria a escolha adequada; para agrupar formas e comparar vetores, o stemming é suficiente e mais barato.

**Ressalva metodológica:** não foi medido o desempenho com lematização. A ablação mostra apenas que remover o stemming derruba a acurácia para 82,4%. Comparar RSLP com um lematizador em português (por exemplo, spaCy) fica como trabalho futuro.

## 4.9 Exemplo completo do pipeline

Pergunta: **"Imagens de vejetassão por drono em 2024"**

| Etapa | Resultado real |
|---|---|
| Normalização | `imagens de vejetassao por drono em 2024` |
| Correção | `imagens de vegetacao por drono em 2024` |
| Canonização de entidades | `imagem de vegetacao por drone em 2024` |
| Canonização de intenção | `imagem de vegetacao por drone em 2024` |
| Números | `imagem de vegetacao por drone em ano` |
| Tokens | `imagem` , `de` , `vegetacao` , `por` , `drone` , `em` , `ano` |
| Stopwords | `imagem` , `vegetacao` , `drone` , `ano` |
| Stems (BOW) | `imag` , `vegetaca` , `dron` , `ano` |
| Entidades | categoria=vegetacao, origem=drone, ano=2024 |
| Intenção | `listar_categoria_origem_ano` |
| SQL | `WHERE categoria=%s AND origem=%s AND EXTRACT(YEAR FROM data_imagem)=%s` |
| Resposta | "Encontrei 2 imagem(ns) de vegetação capturadas por drone de 2024." |

Esse exemplo mostra o ponto mais importante do projeto: **a representação BOW não contém o ano**. Ela contém `ano` . O valor `2024` viaja separadamente e entra na SQL como parâmetro.

---

# 5. Bag of Words, TF-IDF e similaridade de cosseno

## 5.1 Bag of Words

Bag of Words é uma matriz **documento × termo**. Cada linha é um texto; cada coluna é uma palavra do vocabulário; cada célula é a contagem (ou o peso) da palavra naquele texto. A ordem das palavras é ignorada.

Exemplo real com três referências já processadas:

| Documento | agu | dron | imag | list | vegetaca |
|---|---|---|---|---|---|
| "quais imagens possuem vegetacao" | 0 | 0 | 1 | 1 | 1 |
| "mostre imagens de agua" | 1 | 0 | 1 | 1 | 0 |
| "imagens de drone" | 0 | 1 | 1 | 0 | 0 |
| **pergunta: "mostre imagens com vegetacao"** | 0 | 0 | 1 | 1 | 1 |

A pergunta é idêntica à primeira linha, então o cosseno é 1,0. O que o BOW perde: a ordem ("drone de imagens" e "imagens de drone" viram o mesmo vetor) e

os números. Para este problema, a ordem não importa; os números são tratados fora do BOW.

## 5.2 TF-IDF

No BOW puro, toda palavra vale 1. No TF-IDF, cada palavra recebe um peso:

```
peso(t, d) = TF(t, d) × IDF(t)
TF(t, d)   = frequência do termo t no documento d
IDF(t)     = log(N / número de documentos que contêm t)
```

O scikit-learn usa por padrão a variante suavizada:

```
IDF(t) = ln((1 + N) / (1 + df(t))) + 1
```

Palavras que aparecem em quase todos os documentos recebem peso baixo; palavras raras e discriminativas recebem peso alto. No corpus de 134 paráfrases, o vocabulário tem **27 termos**.

| Termo | Documentos que contêm (df) | IDF |
|---|---:|---:|
| `imag` | 128 | 1,0455 |
| `list` | 50 | 1,9734 |
| `cont` | 50 | 1,9734 |
| `ano` | 52 | 1,9350 |
| `vegetaca` | 21 | 2,8142 |
| `dron` | 27 | 2,5731 |
| `satelit` | 25 | 2,6472 |
| `agu` | 16 | 3,0721 |
| `dat` | 7 | 3,8258 |
| `recent` | 7 | 3,8258 |
| `orig` | 7 | 3,8258 |
| `antig` | 4 | 4,2958 |

`imag` aparece em 128 das 134 paráfrases, então quase não discrimina. Já `antig` aparece em apenas 4, por isso tem peso muito maior. Essa é a intuição central do TF-IDF: **o que é comum pesa pouco; o que é raro e distintivo pesa muito**.

Uma decisão importante: o vetorizador é ajustado ( `fit` ) **somente** nas paráfrases de referência. A pergunta do usuário é apenas transformada ( `transform` ). Assim, o vocabulário fica fixo e a comparação é estável.

## 5.3 Similaridade de cosseno

```
cos(A, B) = (A · B) / (||A|| × ||B||)
```

- `A · B` é a soma dos produtos termo a termo;
- `||A||` é a norma euclidiana do vetor;
- o resultado fica entre 0 e 1: 0 significa nenhuma palavra em comum; 1 significa vetores na mesma direção.

O cosseno mede **ângulo**, não distância. Por isso é invariante ao tamanho do texto: uma pergunta curta pode ser comparada a uma referência longa sem ser penalizada pelo comprimento.

Exemplo real com a pergunta `list imag vegetaca`:

| Referência | Cosseno BOW puro | Cosseno TF-IDF |
|---|---|---|
| `list imag vegetaca` | 1,0000 | 1,0000 |
| `list imag agu` | 0,6667 | 0,4811 |
| `imag dron` | 0,4082 | 0,2164 |

Cálculo do segundo caso no BOW puro:

```
A = [0, 0, 1, 1, 1]
B = [1, 0, 1, 1, 0]
A · B = 0+0+1+1+0 = 2
||A|| = √3 ≈ 1,732
||B|| = √3 ≈ 1,732
cos = 2 / (1,732 × 1,732) = 2 / 3 ≈ 0,6667
```

No TF-IDF os valores caem porque `imag` e `list` são palavras comuns e recebem peso menor; a diferença entre as referências fica mais nítida.

## 5.4 Score por intenção

Cada intenção tem várias paráfrases. O score de uma intenção é o **maior cosseno** entre a pergunta e qualquer uma de suas paráfrases.

Exemplo real para "Imagens de drone em 2024":

| Intenção | Score de cosseno |
|---|---|
| `listar_origem_ano` | 1,0000 |
| `listar_origem` | 0,8205 |
| `contar_origem_ano` | 0,7716 |

O motor não escolhe simplesmente o maior score. Ele combina o cosseno com a compatibilidade das entidades extraídas:

- uma intenção que exige um dado ausente é descartada;

- a família contar/listar recebe um ajuste de ±0,18 conforme a pergunta;
- entidades extras que a intenção ignoraria pesam −0,20 cada;
- quando os parâmetros batem exatamente, há um bônus de +0,05;
- palavras como `origem`, `recente`, `antiga` e `data` têm bônus específicos.

No exemplo acima, `listar_origem_ano` tem score 1,0 e ajuste +0,23 (listar +0,18, parâmetros exatos +0,05), resultando em 1,23. O ranqueamento é, portanto, **cosseno + regras de desambiguação**. O limiar de decisão, porém, é aplicado ao cosseno puro.

### 5.5 Limiar e fallback

O limiar padrão é **0,30**. Se o cosseno da intenção vencedora ficar abaixo disso, o sistema responde "não entendi" em vez de arriscar uma SQL errada.

Além do limiar, existe um **filtro de conteúdo**: a pergunta precisa ter pelo menos um stem do domínio:

```
imag, vegetaca, agu, solo_expost, dron, satelit,
orig, ano, dat, recent, antig, tod
```

Há também um filtro para perguntas fora do domínio que usam palavras do nosso vocabulário, como "quanto custa um drone?" (`cust`, `prec`, `valor`, `compr`, `vend`, `alug`).

**Ponto importante para a apresentação:** "Qual a capital da França?" tem cosseno **0,6678** contra `listar_ano`, muito acima de 0,30. O que a rejeita não é o limiar, e sim o filtro de conteúdo: os stems são `list capit franc`, e nenhum deles pertence ao domínio. Isso mostra que o limiar sozinho não basta; o filtro de domínio é a primeira barreira.

### 5.6 TF-IDF versus BOW puro

O projeto compara as duas representações em `demo_bow.py`. No corpus de referência, ambas elegem a intenção correta para o exemplo testado, mas os scores e a separação mudam. Na ablação, o BOW puro (contagem) fica em **89,2%**, enquanto o sistema completo com TF-IDF fica em **100%**. A ponderação ajuda, mas o ganho maior vem da canonização de sinônimos.

---

## 6. Do BOW à SQL: entidades, intenções e catálogo

### 6.1 Extração de entidades

A extração roda sobre o texto **já corrigido**, para que `vejetassao` seja reconhecida como vegetação:

```
def extrair_entidades(texto: str) -> dict:
    texto = normalizar(texto)
    entidades = {}
    # categoria, origem e data/ano
    return entidades
```

Exemplo:

```
"imagens de vegetação por drone em 12/03/2024"
→ {"categoria": "vegetacao", "origem": "drone",
   "data": "2024-03-12", "ano": 2024}
```

Quando há data completa, o ano também é preenchido, mas a data tem prioridade na SQL. O motor remove o ano do conjunto de compatibilidade nesse caso para não penalizar a intenção duas vezes.

## 6.2 Catálogo de intenções

O catálogo tem **20 intenções** e **134 paráfrases**. Cada intenção tem:

```
{
    "id": "listar_categoria",
    "parafrases": ["quais imagens possuem vegetacao", ...],
    "sql": "SELECT ... WHERE categoria = %s ...",
    "tipo": "lista",              # contagem | lista | origens
    "parametros": ["categoria"],  # valores que a SQL precisa
}
```

As famílias são:

| Família | Intenções | Parâmetros |
|---|---|---|
| Contagem | total, categoria, origem, ano e combinações | 0 a 3 |
| Listagem | todas, categoria, origem, ano, data e combinações | 0 a 3 |
| Metadados | origens disponíveis | 0 |
| Ordenação | mais recentes, mais antigas | 0 |

As consultas seguem quatro padrões:

```
-- contagem
SELECT count(*) FROM metadata_table WHERE categoria = %s;

-- listagem com filtro
SELECT id, categoria, data_imagem, origem, caminho_arquivo
FROM metadata_table
WHERE categoria = %s AND origem = %s
ORDER BY data_imagem DESC
LIMIT 60;

-- ano
WHERE EXTRACT(YEAR FROM data_imagem) = %s

-- origens distintas
SELECT DISTINCT origem FROM metadata_table ORDER BY origem;
```

Todas as consultas são **parametrizadas** com `%s`. O texto do usuário nunca é concatenado na SQL; ele entra como parâmetro. Isso elimina SQL injection e mantém a consulta separada dos dados.

### 6.3 O caminho completo de uma pergunta

```
"Imagens de drone em 2024"
        |
        ▼
BOW: imag dron ano
        |
        ▼
TF-IDF + cosseno + regras
        |
        ▼
intenção: listar_origem_ano
entidades: origem=drone, ano=2024
        |
        ▼
SELECT ... WHERE origem = %s AND EXTRACT(YEAR FROM data_imagem) = %s
        |
        ▼
"Encontrei 6 imagem(ns) capturadas por drone de 2024."
```

---

## 7. Robustez a erros de digitação

O requisito de robustez é atendido por **quatro camadas independentes**:

| Camada | O que faz | Exemplo |
|---|---|---|
| Normalização + stemming | absorve acentos, plural e flexão | `imagens` → `imag` |
| Corretor fuzzy | corrige contra léxico com distância de edição | `drono` → `drone` |
| Canonização de sinônimos | mapeia formas equivalentes | `mata` → `vegetacao` |
| Limiar + fallback | evita responder quando não entende | "não entendi" |

Exemplos reais:

| Pergunta com erro | Correção | Intenção |
|---|---|---|
| `quantas imagems existem` | `quantas imagens existem` | `contar_total` |
| `imagens de vejetacao` | `imagens de vegetacao` | `listar_categoria` |
| `fotos de drono` | `fotos de drone` | `listar_origem` |
| `imagens de vejetassao por drono em 2024` | `imagens de vegetacao por drone em 2024` | `listar_categoria_origem_ano` |

A combinação é importante: o corretor resolve o erro explícito; o stemming resolve a variação morfológica; a canonização resolve o sinônimo; e o fallback protege quando nada disso funciona.

**Limitações:** erros na primeira letra não são corrigidos; palavras muito curtas (menos de 4 letras) não passam pelo corretor; e o corretor pode alterar a forma gramatical de palavras que não importam para o BOW.

---

# 8. Avaliação

## 8.1 Conjunto de teste

O conjunto tem **113 perguntas novas**, que não estão no catálogo de referência:

| Grupo | Quantidade | Exemplo |
|---|---:|---|
| Perguntas normais | 92 | "me fala quantas imagens tem no banco" |
| Com erro de digitação | 10 | "imagens de vejetassao por drono em 2024" |
| Fora do domínio | 11 | "qual a capital da França" |

A avaliação mede **a intenção prevista**, não o conteúdo das linhas retornadas. Isso é adequado para comparar o pipeline de PLN, mas não substitui um teste de resultado SQL.

## 8.2 Métricas

| Métrica | Resultado |
|---|---:|
| Acurácia geral | 100% (113/113) |
| Acurácia em perguntas normais | 100% (92/92) |
| Acurácia com erro de digitação | 100% (10/10) |
| Rejeição de perguntas fora do domínio | 100% (11/11) |
| Taxa de fallback | 9,7% |

**Observação metodológica:** o conjunto foi escrito pela equipe durante o desenvolvimento, não é um benchmark cego. Ele serve para comparar as etapas do pipeline e justificar decisões. Perguntas muito diferentes das paráfrases podem ter desempenho inferior.

## 8.3 Ablação

A ablação desliga uma etapa por vez e mede o impacto somente nas perguntas do domínio:

| Configuração | Acurácia |
|---|---|
| Sistema completo (com regras) | 100,0% |
| Sem regras de entidades | 82,4% |
| Sem normalização | 82,4% |
| Sem stopwords | 71,6% |
| Sem stemming | 82,4% |
| Sem substituição de números | 73,5% |
| Sem canonização de sinônimos | 55,9% |
| Sem corretor ortográfico | 79,4% |
| Sem TF-IDF (BOW puro) | 89,2% |

Leitura dos números:

- **canonização** é a etapa mais crítica (55,9% sem ela);
- **stopwords** e **números** também são essenciais;
- o **corretor** contribui, mas o conjunto tem apenas 10 casos com erro;
- o **BOW puro** funciona razoavelmente, mas o TF-IDF separa melhor.

## 8.4 Curva do limiar

| Limiar | Acurácia geral | Acurácia domínio | Fallback | OOD rejeitado |
|---|---|---|---|---|
| 0,10 | 100,0% | 100,0% | 9,7% | 100,0% |
| 0,20 | 100,0% | 100,0% | 9,7% | 100,0% |
| 0,30 | 100,0% | 100,0% | 9,7% | 100,0% |
| 0,40 | 100,0% | 100,0% | 9,7% | 100,0% |
| 0,50 | 100,0% | 100,0% | 9,7% | 100,0% |
| 0,60 | 100,0% | 100,0% | 9,7% | 100,0% |

**Leitura honesta:** a curva é plana porque o conjunto de teste não estressa o limiar. Quem rejeita as perguntas fora do domínio é o filtro de conteúdo, não o limiar. O limiar de 0,30 é uma segunda barreira, útil para perguntas do domínio

que sejam muito distantes das paráfrases. Para justificar melhor esse valor, seria necessário um conjunto com perguntas ambíguas do próprio domínio.

## 8.5 Por que o limiar é 0,30?

O limiar é aplicado ao **cosseno puro** entre a pergunta e a melhor paráfrase da intenção vencedora. O score ajustado (cosseno + regras de entidades) serve para ordenar candidatos, mas não para decidir se o sistema responde ou não.

A escolha de 0,30 se apoia em cinco argumentos:

1. **Faixa usual em recuperação textual com TF-IDF.** Em problemas de similaridade lexical com perguntas curtas, limiares entre 0,25 e 0,40 são comuns. Abaixo disso, o sistema aceita correspondências fracas; muito acima, rejeita perguntas válidas que usam palavras diferentes das paráfrases.
2. **Equilíbrio entre cobertura e precisão.** Com 0,30, o sistema responde às perguntas do domínio e mantém o fallback em 9,7% no conjunto de teste — os 11 casos fora do domínio. Baixar para 0,10–0,20 aumentaria o risco de aceitar perguntas vagas; subir para 0,50–0,60 aumentaria o risco de "não entendi" em perguntas válidas, sobretudo as que têm poucas palavras em comum com as paráfrases.
3. **O limiar não é a única barreira.** Antes dele, o sistema exige pelo menos um stem de conteúdo do domínio (`imag`, `vegetaca`, `agu`, `solo_expost`, `dron`, `satelit`, `ano`, `dat`, `recent`, `antig`, `tod`) e rejeita perguntas com vocabulário de preço/compra. É esse filtro que rejeita "Qual a capital da França?", cujo cosseno é 0,6678. O limiar de 0,30 atua como segunda barreira para perguntas do domínio que sejam muito distantes das paráfrases.
4. **A curva do limiar é plana no conjunto atual.** Os valores de 0,10 a 0,60 produzem a mesma acurácia (100%) e o mesmo fallback (9,7%). Isso mostra que o conjunto não é sensível ao limiar; 0,30 é uma escolha conservadora e documentada, não um valor ajustado por sobreposição ao teste. Em um conjunto mais difícil, a calibração deveria ser refeita.
5. **Custo assimétrico do erro.** Neste trabalho, optamos por uma postura conservadora: preferir o fallback a executar uma SQL com a intenção errada. O limiar de 0,30 e o filtro de conteúdo implementam essa preferência.

Em resumo: 0,30 é um valor de partida defensável para TF-IDF + cosseno, mantém 100% de acerto no conjunto atual e evita respostas frágeis. O próprio resultado da curva mostra que o valor pode ser reavaliado com um conjunto de validação mais difícil; a decisão está registrada em `.env` (`LIMIAR=0.30`) e é facilmente ajustável sem alterar o código.

## 8.6 Análise de erro

No conjunto atual não há erros de classificação. As limitações aparecem em casos que não estão no conjunto, como:

- duas categorias na mesma pergunta ("vegetação e água");
- dois anos na mesma pergunta ("2023 e 2024");
- meses ("janeiro de 2024");

- combinações fora do catálogo ("imagem mais recente de vegetação").

Neste último caso, o motor escolhe `mais_recentes` e **ignora a categoria**, porque essa intenção não tem parâmetro de categoria. É uma limitação inerente ao catálogo, não um bug do cosseno.

---

# 9. Limitações e trabalhos futuros

## 9.1 Limitações

1. **Busca lexical, não semântica.** Sinônimos só funcionam se estiverem no dicionário. O sistema não infere relações de sentido.
2. **Uma categoria e um ano por pergunta.** O extrator pega a primeira ocorrência; "vegetação e água" ou "2023 e 2024" não são combinados.
3. **Meses não são filtrados.** "janeiro de 2024" filtra apenas pelo ano.
4. **Combinações fora do catálogo** podem ignorar um filtro, como no exemplo "imagem mais recente de vegetação".
5. **Conteúdo visual não é analisado.** A busca é sobre metadados; o conteúdo da imagem não entra no BOW.
6. **Avaliação não cega.** O conjunto de 113 perguntas foi escrito pela equipe.
7. **Curva do limiar plana.** O conjunto não exercita o limiar; o filtro de conteúdo domina a rejeição de perguntas fora do domínio.
8. **Bug conhecido na extração por substring.** A verificação `if s in texto` faz `"periodo"` conter `"rio"` e ser classificado como água; `"marrom"` contém `"mar"` e também vira água. A correção é usar limites de palavra (`\b`) como já é feito na canonização.
9. **Imagens sintéticas.** O `seed.py` gera imagens de exemplo com PIL; o conteúdo visual não afeta o resultado.

## 9.2 Melhorias possíveis

- corrigir a extração de entidades com regex de palavra completa;
- adicionar intenções para múltiplas categorias, múltiplos anos e meses;
- separar um conjunto de desenvolvimento e um conjunto de teste cego;
- avaliar também o resultado das SQLs, não só a intenção;
- testar n-gramas de caracteres (`char_wb`, 3 a 5) como alternativa ao corretor;
- permitir desambiguação quando duas intenções empatam;
- expor no painel de debug a lista de candidatos descartados e o motivo.

---

# 10. Conclusão

O projeto atende ao enunciado: aplicação de página única em Python/Flask com PostgreSQL, pipeline clássico de PLN, representação BOW, similaridade de cosseno, mapeamento para SQL e robustez a erros de digitação.

O diferencial não é a interface, e sim a **transparência do pipeline**. O painel "Mostrar como o sistema pensou" exibe a normalização, a correção, a canonização, os tokens, os stems, o BOW, o ranking de intenções e a SQL executada. Isso torna o comportamento auditável e permite explicar cada decisão.

O sistema é:

- **interpretável**: mostra por que escolheu uma intenção;
- **offline**: não depende de API paga nem de LLM;
- **determinístico**: mesma pergunta, mesmo resultado;
- **reprodutível**: `./iniciar.sh` sobe banco, dados e site.

A principal lição de PLN é que a qualidade não vem de um modelo gigante, mas da combinação cuidadosa de etapas simples: normalizar, corrigir, canonizar, remover ruído, reduzir à raiz e comparar vetores. Cada etapa resolve um tipo de variação linguística, e a ablação mostra exatamente quanto cada uma contribui.

---

## Apêndice A — Como reproduzir

```
./iniciar.sh

# em outro terminal
source .venv/bin/activate
python demo_pln.py
python demo_bow.py
python testar_parafrases.py
python avaliacao.py
```

O site fica em `http://127.0.0.1:5000`. O banco PostgreSQL sobe em Docker na porta 5433 (container `pgsem`, banco `imagens_db`).

Teste rápido da API:

```
curl -s -X POST http://127.0.0.1:5000/api/busca \
  -H "Content-Type: application/json" \
  -d '{"pergunta": "quantas imagens existem?"}'
```

## Apêndice B — Arquivos do núcleo de PLN

| Arquivo | Função |
|---|---|
| `texto.py` | normalização |
| `corretor.py` | correção ortográfica com distância de edição |
| `lexico.py` | sinônimos de intenção |
| `entidades.py` | sinônimos de entidade e extração de valores |
| `pln.py` | orquestra o pipeline e gera o BOW |

| Arquivo | Função |
|---|---|
| `intencoes.py` | 20 intenções, 134 paráfrases e SQL |
| `motor.py` | TF-IDF, cosseno, ranking e execução da SQL |
| `app.py` | API Flask |
| `avaliacao.py` | métricas, ablação e curva do limiar |

## Apêndice C — Roteiro de 7 minutos

1. **Abertura (40 s):** "É um motor de busca com PLN clássico: normalização, tokenização, stopwords, stemming, BOW e cosseno. Não usa LLM."
2. **Contagem (1 min):** clique em "Quantas imagens existem?" e mostre `select count(*)` e a resposta "No total, há 60 imagens cadastradas."
3. **Listagem por categoria (1 min):** clique em "Quais imagens possuem vegetação?" e mostre os cards.
4. **Filtro combinado (1 min):** "Imagens de drone em 2024" e explique que o BOW decide a intenção e a entidade fornece o valor.
5. **Erro de digitação (1 min):** "Imagens de vejetassão por drono em 2024" e explique as camadas de robustez.
6. **Debug (1,5 min):** abra "Mostrar como o sistema pensou" e mostre tokens, stems, top 3 scores e a SQL.
7. **Fora do domínio (1 min):** "Qual a capital da França?" e explique o filtro de conteúdo e o fallback.
8. **Fechamento (30 s):** "Não é IA generativa; é PLN clássico, interpretável, offline e reprodutível."